

Тестовое задание

Онлайн-библиотека (FS)

Идея: Разработать полноценное веб-приложение — **Онлайн-библиотеку**, где пользователи могут искать книги через открытое API (Open Library), оставлять комментарии, ставить лайки, формировать список для чтения с разными статусами, управлять своим профилем.

Функциональные требования (User Stories):

Гостевой доступ (неавторизованный пользователь)

1. **Регистрация и вход.** Я могу зарегистрироваться в приложении (указать логин и пароль) и войти. После входа я получаю JWT-токен, который хранится на клиенте (localStorage или httpOnly cookie — на усмотрение кандидата).
2. **Поиск книг.** Будучи гостем, я могу искать книги через Open Library API по ключевым словам (название, автор) и просматривать результаты в виде карточек (название, автор, обложка).
3. **Просмотр деталей книги.** Будучи гостем, я могу перейти на страницу конкретной книги и увидеть:
 - Название, авторов, описание, обложку
 - Количество лайков (от зарегистрированных пользователей)
 - Список публичных комментариев с указанием авторов и дат

Авторизованный пользователь (дополнительно к возможностям гостя)

4. **Лайки.** Я могу поставить книге лайк и убрать лайк. Количество лайков под книгой обновляется в реальном времени и видно всем пользователям (включая гостей).
5. **Комментарии.** Я могу написать публичный комментарий к любой книге. Я могу редактировать или удалять **только свои** комментарии. Комментарии видны всем (гостям и пользователям)

6. **Список для чтения со статусами.** Я могу добавить книгу в свой список для чтения, выбрав один из статусов:

- «Хочу прочитать»
- «Читаю сейчас»
- «Прочитано»

Я могу изменить статус книги или удалить её из списка.

7. **Просмотр моего списка для чтения.** Я могу видеть все книги, которые я добавил в список, с указанием их текущего статуса (и фильтровать по статусу).

8. **Просмотр моих лайков.** Я могу видеть список книг, которым я поставил лайк.

9. **Поиск по моим книгам.** Я могу искать книги (только среди своих) — либо среди тех, что я лайкнул, либо среди тех, что находятся в моём списке для чтения (с возможностью фильтрации по этим двум категориям).

10. **История моих комментариев.** Я могу просматривать историю всех своих комментариев (с указанием книги, даты, текста) и оттуда переходить к редактированию/удалению.

11. **Профиль пользователя.** Я могу видеть в профиле свой логин. Я могу изменить логин (если новый логин - не занят) или пароль (предварительно введя текущий пароль).

Технические требования

Общие требования

- **Node.js** (LTS) — версия указывается в `.nvmrc`
- **TypeScript** (и бэкенд, и фронтенд)
- **Бэкенд:** Express.js
- **Фронтенд:** React (Create React App или Vite) + React Router
- **Стилизация:** CSS, SCSS, Tailwind или другой произвольный CSS-фреймворк - на выбор кандидата
- **База данных:** Supabase (PostgreSQL) или Firebase Firestore - на выбор кандидата
- **HTTP-клиент:** Axios
- **Документация API:** Swagger (OpenAPI 3.0)

Авторизация и безопасность

- JWT (access token) для авторизации. Токен передаётся в заголовке `Authorization: Bearer <token>`
- Пароли хранятся в БД только в хэшированном виде (bcrypt или аналоги)
- При регистрации проверять уникальность логина

Работа с Open Library API

- **Поиск книг:** <https://openlibrary.org/search.json?q=...>
- **Детали книги:** <https://openlibrary.org/works/{olid}.json>
- **Обложки:**
https://covers.openlibrary.org/b/olid/{cover_edition_key}-L.jpg (размеры S/M/L)
- **Лимиты API:** следует соблюдать не более 1 запроса в секунду (реализовать очередь или throttle при необходимости)

Кеширование

- Все запросы к **внешнему API** (поиск, получение деталей книги) должны кешироваться **в памяти сервера**
- **Время жизни кеша:** 30 минут
- Кеш должен работать по параметрам запроса (например, для поиска — ключ `q`, для деталей — `olid`).
- **Рекомендуемые npm-библиотеки для in-memory кеширования:**
 - `node-cache` (<https://www.npmjs.com/package/node-cache>)
 - `memory-cache` (<https://www.npmjs.com/package/memory-cache>)
 - `lru-cache` (<https://www.npmjs.com/package/lru-cache>)
- Повторные запросы от одного и того же пользователя с теми же параметрами в течение 30 минут не должны вызывать повторного обращения к Open Library API (т.е. кешируются только вызовы к Open Library, а свои списки всегда свежие из БД)

Хранение пользовательских данных

В БД должны быть реализованы следующие сущности (кандидат проектирует схему самостоятельно, но она должна покрывать все user stories):

- Пользователи (логин, хэш пароля, дата регистрации, возможность смены логина/пароля)
- Лайки (связь пользователь-книга) — книга идентифицируется по `olid` (Open Library ID)
- Комментарии (текст, автор, книга, дата создания/обновления)
- Список чтения (пользователь, книга, статус из трёх возможных)

Примечание: сами книги не хранятся в БД, только их идентификаторы ([olid](#)). Для отображения списков (лайков, чтения) кандидат может кешировать базовые поля (название, автор, обложку) в своей БД или запрашивать их из внешнего API (с учётом кеширования). Выбор стратегии остаётся за кандидатом.

Требования к фронтенду (React)

1. Все user stories должны быть реализованы в виде отдельных страниц/компонентов.
2. Обязательные страницы/блоки:
 - a. Регистрация / Вход
 - b. Главная страница с поиском (карточки книг)
 - c. Страница деталей книги (лайки, комментарии, добавление в список чтения)
 - d. Личный кабинет:
 - i. Профиль (смена логина/пароля)
 - ii. Мои лайки (список книг)
 - iii. Мой список для чтения (с фильтром по статусам)
 - iv. История комментариев (с возможностью редактирования/удаления)
 - v. Поиск по моим книгам (по лайкам или списку чтения)
3. Обработка состояний загрузки (loading) и ошибок (error)
4. Пагинация на всех списках (поиск, лайки, список чтения, комментарии)
5. Везде, где есть информация по книге (поиск, список лайков, детали о книге и т.д.) - обязательно должна отображаться и обложка книги
6. Маршрутизация с защитой приватных страниц (только для авторизованных пользователей)

Использование AI

Кандидат может (и ему рекомендуется) использовать инструменты на базе искусственного интеллекта (например, GitHub Copilot, ChatGPT, Cursor и т.п.) для ускорения разработки, генерации шаблонного кода, написания тестов, документации и т.д.

Предоставление проекта

Работа ведётся в **публичном GitHub-репозитории** с названием `online-library-<surname>`, где вместо `<surname>` следует подставить свою фамилию на латинице

Структура проекта:

- `backend/` и `frontend/` — папки с исходным кодом бэкенда и фронтенда соответственно. В каждой из них:
 - `.env.example` — шаблон переменных окружения (порты, URL БД, JWT secret и т.д.)
 - `.nvmrc` — версия Node.js (например, `v24.1.0`)
 - `.eslintrc` и `.prettierrc` — конфиги линтеров
 - `README.md` — инструкция по запуску (бэкенд + фронтенд), краткое описание архитектуры, ссылки на Swagger
- Папка `docs/`:
 - содержит один `results.docx` со скриншотами:
 - Скриншоты всех основных страниц (гость + пользователь)
 - Скриншоты работы API (Swagger или Postman) для ключевых эндпоинтов
 - Демонстрация работы кеширования (например, повторный запрос поиска не вызывает обращения к Open Library)
 - файл `AI_USAGE.md`, в котором кратко описать:
 - какие AI-инструменты применялись
 - для каких задач (генерация компонентов, написание SQL-схем, отладка, рефакторинг, создание Swagger-документации и пр.)

- как это повлияло на процесс (что ускорило, с чем помогло справиться)
- контент файл не оценивается как «плюс/минус», а служит для понимания подхода кандидата к современным средствам разработки
- ER-диаграмма (схема БД) `database.jpeg` (допустим любой другой формат картинки или `.drawio`)
- Скриншоты покрытия тестами (если есть)

Критерии оценки

1. **Полнота реализации user stories** — все описанные сценарии работают корректно
2. **Архитектура и чистота кода** — разделение на контроллеры, сервисы, репозитории; использование TypeScript; читаемость.
3. **Качество API** — корректные HTTP-статусы, валидация входных данных, обработка ошибок внешнего API
4. **Работа с БД** — правильные схемы, индексы, уникальность (например, пользователь не может лайкнуть книгу дважды)
5. **Кеширование** — реализовано in-memory кеширование внешних запросов на 30 минут, повторные запросы не дублируются
6. **Фронтенд** — все страницы работают, управление состоянием (React hooks), обработка асинхронных запросов, удобный интерфейс
7. **Документация** — Swagger с описанием всех эндпоинтов, понятный README, скриншоты в `demo/`
8. **Безопасность** — хэширование паролей, JWT, защита от SQL-инъекций (через ORM/SDK)
9. **Работа с git** — репозиторий не содержит лишних громоздких папок/файлов (например, `node_modules`), атомарные коммиты, стабильность коммитов

Бонусные баллы (не обязательно, но приветствуется):

- Unit-тесты (на сервисы бэкенда) и/или e2e-тесты (Cypress)
- Throttling запросов к Open Library API (ограничение частоты)

Удачи! Ждём ваше решение